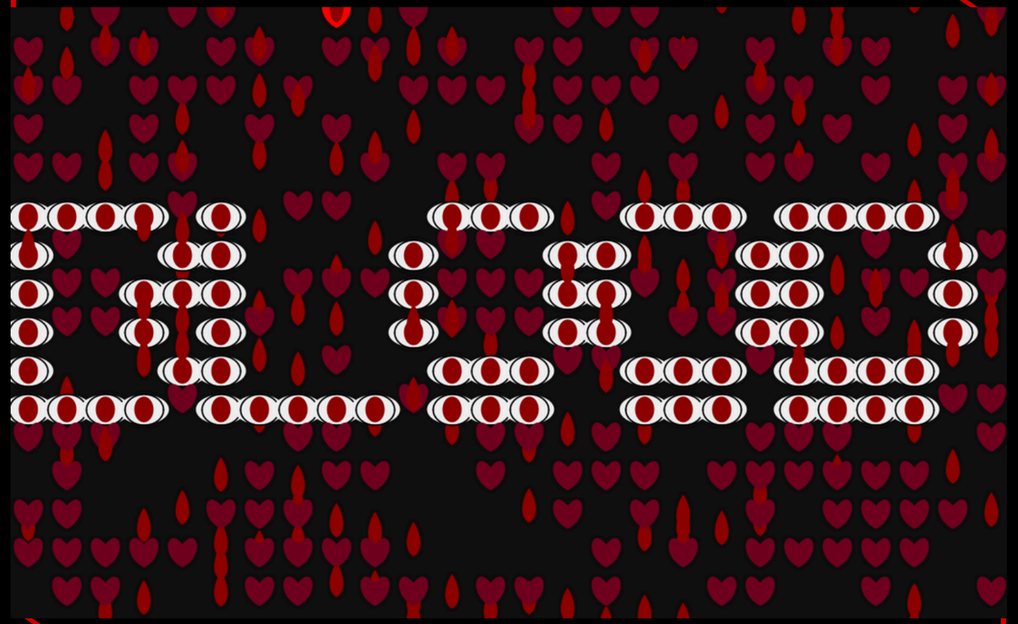
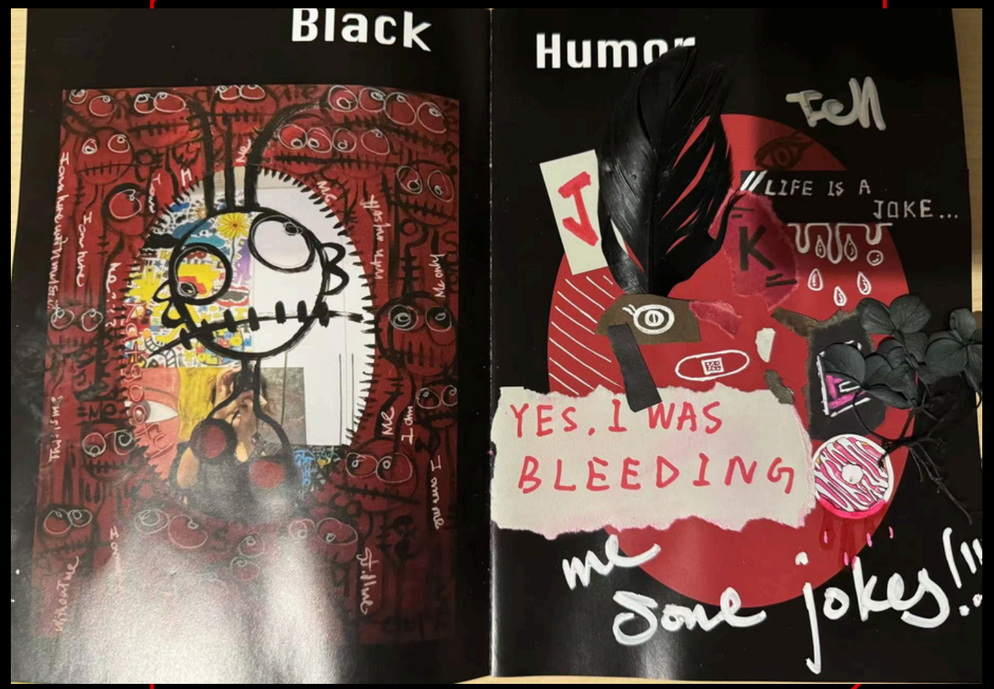
Design Journal

BLEEDING MOOD



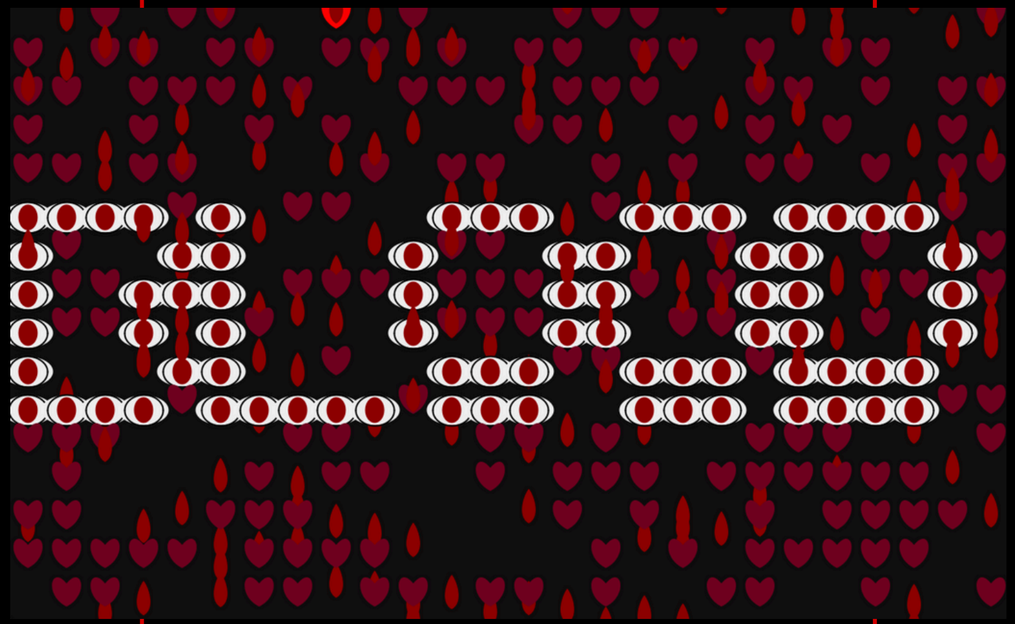
Yihong Lin

CS208-HW2



Inspiration

This design was inspired by an art workshop I attended before, I made my own creative book with various tools and materials. The topic I chose was Bleeding mood, which represents the hurt and negative emotions we suffered in our life.
People always encounter many unpleasant things and frustrations in life, which can also bring negative mood and state. However, we should try to accept our negative emotions, allow ourselves to be hurt and sad, and protect our Bleeding mood more



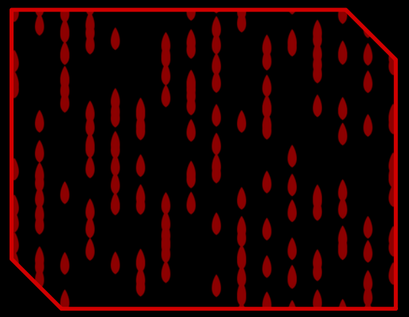
Concept

I used the simple elements of love, water drops, and eyes. The drop of water represents the tear, the love represents the heart and feelings, and the eyes represent the gaze. Drops of water drop like tears, pupils and whites of the eyes spell out the word BLOOD as a background, and other places are randomly distributed in the shape of love hearts.

When the mouse hovers over the pupil, the pupil turns blue and black, indicating the pupil of a person in a normal mood. But what I want to express is that there are more unpleasant things in life than good things, so you should be calm and accept setbacks.

When the mouse hovers over the love, it will turn bright red, as if the heart is restored to life, but with the mouse click on the love, it will display the appearance of heartbreak, indicating the hurt mood.

Generative rules



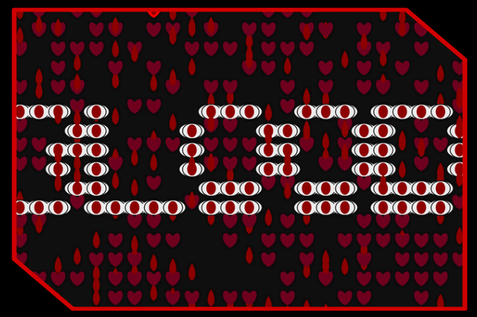
Grid System

int gridSize = 50;
int numCols, numRows;
numCols = width / gridSize;
numRows = height / gridSize;

```
for (int i = 0; i < numCols; i++) {
  for (int j = 0; j < numRows; j++) {
    if (bloodPattern[j][i]) {
      modules.add(new EyeModule(i * gridSize, j * gridSize, gridSize));
    } else {
      int moduleType = int(random(2));
      switch (moduleType) {
        case 0:
          modules.add(new DropModule(i * gridSize, j * gridSize, gridSize));
          break;
        case 1:
          modules.add(new HeartModule(i * gridSize, j * gridSize, gridSize));
          break;
      }
    }
```

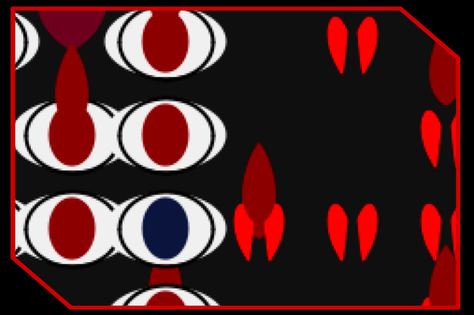
Module Design

- EyeModule:
  - Represents the characters of "BLOOD".
- DropModule:
  - Represents falling drops of tear.
  - Used to fill the background randomly.
- HeartModule:
  - Represents a heart shape.
  - Design is a simple heart shape that can break into two halves when clicked.
  - Used to fill the background randomly.



Tiling Method

- Character Representation:
  - The word "BLOOD" is represented using EyeModule instances.
  - Each character is defined by a 2D boolean array where true indicates the placement of an EyeModule.
- Background Filling:
  - The background (areas not part of "BLOOD") is filled with HeartModule and DropModule instances.
  - The type of module (heart or drop) is chosen randomly for each grid cell not part of the "BLOOD" pattern.



Animations / Interactions

- DropModule Animation:
  - DropModule instances have a simple falling animation.
  - They move downwards at a speed determined by the speed variable.
  - When a DropModule reaches the bottom of the canvas, it reappears at the top.
- HeartModule Interaction:
  - When your mouse hovers over the love, it will turn bright red.
  - When a HeartModule is clicked, it breaks into two halves.
  - The broken state is represented by two separate half-heart shapes.
- EyeModule Interaction:
  - When your mouse hovers over the pupils, it will turn dark blue.

/Code

```
int numCols, numRows;
ArrayList<PatternModule> modules;

void setup() {
  size(1300, 800);
  numCols = width / gridSize;
  numRows = height / gridSize;
  modules = new ArrayList<PatternModule>();
  initModules();
}

void draw() {
  background(15);
  for (PatternModule module : modules) {
    module.display();
    module.update();
```

Grid System

```
for (int i = 0; i < numCols; i++) {
  for (int j = 0; j < numRows; j++) {
    if (bloodPattern[j][i]) {
      modules.add(new EyeModule(i * gridSize, j * gridSize, gridSize));
    } else {
      int moduleType = int(random(2));
      switch (moduleType) {
        case 0:
          modules.add(new DropModule(i * gridSize, j * gridSize, gridSize));
          break;
        case 1:
          modules.add(new HeartModule(i * gridSize, j * gridSize, gridSize));
          break;
      }
    }
```

Module Design

- eye (display code here) ⟶
- drop
- heart(display code here)

```
  isBroken = false;
}

void display() {

  if (isHovered) {
    fill(255, 0, 0);
  } else {
    fill(104, 15, 36);
  }
  noStroke();

  if (isBroken) {
    drawBrokenHeart();
  } else {
    drawCompleteHeart();
  }
}

void drawCompleteHeart() {
  beginShape();
  vertex(x + size / 2, y);


  bezierVertex(x + size * 0.9, y - size * 0.5, x + 1.1 * size, y + size * 0.3, x + size / 2, y + size * 0.6);

  vertex(x + size / 2, y + size * 0.6);

  bezierVertex(x - size * 0.1, y + size * 0.3, x + size * 0.1, y - size * 0.5, x + size / 2, y);
  endShape(CLOSE);
}

void drawBrokenHeart() {
  noStroke();
  fill(255, 0, 0);

  beginShape();
  vertex(x + size / 2 - size * 0.05, y + size * 0.4);
  bezierVertex(x + size * 0.3, y, x + size * 0.1, y + size * 0.4, x + size * 0.35, y + size * 0.8);
  bezierVertex(x + size * 0.4, y + size * 0.9, x + size / 2 - size * 0.1, y + size * 0.9, x + size / 2 - size * 0.05, y + size * 0.4
  );
```

Tiling Method

- use boolean to draw

```
boolean isHovered;

EyeModule(float x, float y, float size) {
  super(x, y, size);
  isHovered = false;
}

void display() {

  noStroke();
  fill(240);
  ellipse(x + size / 2, y + size / 2, size * 1.3, size * 0.77);


  if (isHovered) {
    fill(15, 20, 60);
  } else {
    fill(128, 0, 0);
  }
  ellipse(x + size / 2, y + size / 2, size / 2, size / 1.5);


  stroke(0);
  strokeWeight(2);
  noFill();
  arc(x + size / 2, y + size / 2, size * 1, size * 0.8, PI, TWO_PI);
  arc(x + size / 2, y + size / 2, size * 1, size * 0.8, 0, PI);
}

void update() {
  if (mouseX > x && mouseX < x + size && mouseY > y && mouseY < y + size) {
    isHovered = true;
  } else {
    isHovered = false;
```

⟶

```
boolean isHovered;
boolean isBroken;

HeartModule(float x, float y, float size) {
  super(x, y, size);
  isHovered = false;
  isBroken = false;
}

void display() {

  if (isHovered) {
    fill(255, 0, 0);
  } else {
    fill(104, 15, 36);
  }
  noStroke();

  if (isBroken) {
    drawBrokenHeart();
  } else {
    drawCompleteHeart();
  }
}

void drawCompleteHeart() {
  beginShape();
  vertex(x + size / 2, y);


  bezierVertex(x + size * 0.9, y - size * 0.5, x + 1.1 * size, y + size * 0.3, x + size / 2, y + size * 0.6);

  vertex(x + size / 2, y + size * 0.6);

  bezierVertex(x - size * 0.1, y + size * 0.3, x + size * 0.1, y - size * 0.5, x + size / 2, y);
  endShape(CLOSE);
}

void drawBrokenHeart() {
  noStroke();
  fill(255, 0, 0);

  beginShape();
  vertex(x + size / 2 - size * 0.05, y + size * 0.4);
  bezierVertex(x + size * 0.3, y, x + size * 0.1, y + size * 0.4, x + size * 0.35, y + size * 0.8);
  bezierVertex(x + size * 0.4, y + size * 0.9, x + size / 2 - size * 0.1, y + size * 0.9, x + size / 2 - size * 0.05, y + size * 0.4);
```

Animation

- use speed

```
class DropModule extends PatternModule {
  float speed;

  DropModule(float x, float y, float size) {
    super(x, y, size);
    speed = random(1, 3);
  }

  void display() {
    noStroke();
    fill(128, 0, 0);
    beginShape();

    vertex(x + size / 2, y + size);

    bezierVertex(x + size * 0.85, y + size * 0.8

    bezierVertex(x + size * 0.4, y + size * 0.2,
    endShape(CLOSE);
  }

  void update() {
    y += speed;
    if (y > height) {
      y = -size;
    }
```

```
void update() {
  if (mouseX > x && mouseX < x + size && mouseY > y && mouseY < y + size) {
    isHovered = true;
    if (mousePressed) {
      isBroken = true;
    }
  } else {
    isHovered = false;
  }
}
```

Interactions

- use true/false

```
HeartModule(float x, float y, float siz
  super(x, y, size);
  isHovered = false;
  isBroken = false;
}

void display() {

  if (isHovered) {
    fill(255, 0, 0);
  } else {
    fill(104, 15, 36);
  }
  noStroke();

  if (isBroken) {
    drawBrokenHeart();
  } else {
    drawCompleteHeart();
  }
```

Interactions

- use isHovered

Code

```
int gridSize = 50;
int numCols, numRows;
ArrayList<PatternModule> modules;

void setup() {
  size(1300, 800);
  numCols = width / gridSize;
  numRows = height / gridSize;
  modules = new ArrayList<PatternModule>();
  initModules();
}

void draw() {
  background(15);
  for (PatternModule module : modules) {
    module.display();
    module.update();
  }
}

void initModules() {

  boolean[][] bloodPattern = new boolean[numRows][numCols];
  String blood = "BLOOD";
  int charWidth = blood.length() * 5;
  int startCol = (numCols - charWidth) / 2;
  int startRow = numRows / 2 - 3;


  for (int i = 0; i < blood.length(); i++) {
    char c = blood.charAt(i);
    switch (c) {
      case 'B':
        fillBloodPattern(bloodPattern, startCol + i * 5, startRow, 'B');
        break;
      case 'L':
        fillBloodPattern(bloodPattern, startCol + i * 5, startRow, 'L');
        break;
      case 'O':
        fillBloodPattern(bloodPattern, startCol + i * 5, startRow, 'O');
        break;
      case 'D':
        fillBloodPattern(bloodPattern, startCol + i * 5, startRow, 'D');
        break;
    }
  }


  for (int i = 0; i < numCols; i++) {
    for (int j = 0; j < numRows; j++) {
      if (bloodPattern[j][i]) {
        modules.add(new EyeModule(i * gridSize, j * gridSize, gridSize));
      } else {
```

```
    noStroke();
    fill(240);
    ellipse(x + size / 2, y + size / 2, size * 1.3, size * 0.77);


    if (isHovered) {
      fill(15, 20, 60);
    } else {
      fill(128, 0, 0);
    }
    ellipse(x + size / 2, y + size / 2, size / 2, size / 1.5);


    stroke(0);
    strokeWeight(2);
    noFill();
    arc(x + size / 2, y + size / 2, size * 1, size * 0.8, PI, TWO_PI);
    arc(x + size / 2, y + size / 2, size * 1, size * 0.8, 0, PI);
  }

  void update() {
    if (mouseX > x && mouseX < x + size && mouseY > y && mouseY < y + size) {
      isHovered = true;
    } else {
      isHovered = false;
    }
  }
}


class DropModule extends PatternModule {
  float speed;

  DropModule(float x, float y, float size) {
    super(x, y, size);
    speed = random(1, 3);
  }

  void display() {
    noStroke();
    fill(128, 0, 0);
    beginShape();

    vertex(x + size / 2, y + size);

    bezierVertex(x + size * 0.85, y + size * 0.8, x + size * 0.6, y + size * 0.2, x + size / 2, y + size * 0.1)

    bezierVertex(x + size * 0.4, y + size * 0.2, x + size * 0.15, y + size * 0.8, x + size / 2, y + size);
```

```
en;

oat x, float y, float size) {
size);
false;
alse;


 {

d) {
0, 0);
  } else {
    fill(104, 15, 36);
  }
  noStroke();

  if (isBroken) {
    drawBrokenHeart();
  } else {
    drawCompleteHeart();
  }
}

void drawCompleteHeart() {
  beginShape();
  vertex(x + size / 2, y);


  bezierVertex(x + size * 0.9, y - size * 0.5, x + 1.1 * size, y + size * 0.3, x + size / 2, y + size * 0.6);

  vertex(x + size / 2, y + size * 0.6);

  bezierVertex(x - size * 0.1, y + size * 0.3, x + size * 0.1, y - size * 0.5, x + size / 2, y);
  endShape(CLOSE);
}

void drawBrokenHeart() {
  noStroke();
  fill(255, 0, 0);

  beginShape();
  vertex(x + size / 2 - size * 0.05, y + size * 0.4);
  bezierVertex(x + size * 0.3, y, x + size * 0.1, y + size * 0.4, x + size * 0.35, y + size * 0.8);
  bezierVertex(x + size * 0.4, y + size * 0.9, x + size / 2 - size * 0.1, y + size * 0.9, x + size / 2 - size * 0.05, y + size * 0.4);
  endShape();


  beginShape();
  vertex(x + size / 2 + size * 0.05, y + size * 0.4);
  bezierVertex(x + size * 0.7, y, x + size * 0.9, y + size * 0.4, x + size * 0.65, y + size * 0.8);
  bezierVertex(x + size * 0.6, y + size * 0.9, x + size / 2 + size * 0.1, y + size * 0.9, x + size / 2 + size * 0.05, y + size * 0.4);
  endShape();
```

```
  else if (c == 'D') {
    setPatternValue(pattern, startRow, startCol + 1);
    setPatternValue(pattern, startRow, startCol );
    setPatternValue(pattern, startRow, startCol + 2);
    setPatternValue(pattern, startRow, startCol + 3);
    setPatternValue(pattern, startRow + 1, startCol);
    setPatternValue(pattern, startRow + 1, startCol + 4);
    setPatternValue(pattern, startRow + 2, startCol);
    setPatternValue(pattern, startRow + 2, startCol + 4);
    setPatternValue(pattern, startRow + 3, startCol);
    setPatternValue(pattern, startRow + 3, startCol + 4);
    setPatternValue(pattern, startRow + 4, startCol);
    setPatternValue(pattern, startRow + 4, startCol + 1);
    setPatternValue(pattern, startRow + 4, startCol + 2);
    setPatternValue(pattern, startRow + 4, startCol + 3);
    setPatternValue(pattern, startRow + 5, startCol );
    setPatternValue(pattern, startRow + 5, startCol + 1);
    setPatternValue(pattern, startRow + 5, startCol + 2);
    setPatternValue(pattern, startRow + 5, startCol + 3);

  }
}

void setPatternValue(boolean[][] pattern, int row, int col) {
  if (row >= 0 && row < pattern.length && col >= 0 && col < pattern[0].length) {
    pattern[row][col] = true;
  }
}


abstract class PatternModule {
  float x, y, size;

  PatternModule(float x, float y, float size
    this.x = x;
    this.y = y;
    this.size = size;
  }

  abstract void display();
  abstract void update();
}

class EyeModule extends PatternModule {
  boolean isHovered;

  EyeModule(float x, float y, float size) {
    super(x, y, size);
    isHovered = false;
  }

  void display() {
```

```
      } else {
        int moduleType = int(random(2));
        switch (moduleType) {
          case 0:
            modules.add(new DropModule(i * gridSize, j * gridSize, gridSize));
            break;
          case 1:
            modules.add(new HeartModule(i * gridSize, j * gridSize, gridSize));
            break;
        }
      }
    }
  }
}

void fillBloodPattern(boolean[][] pattern, int startCol, int startRow, char c) {
  if (c == 'B') {

    setPatternValue(pattern, startRow, startCol + 2);
    setPatternValue(pattern, startRow, startCol + 3);
    setPatternValue(pattern, startRow, startCol + 1);
    setPatternValue(pattern, startRow + 1, startCol );
    setPatternValue(pattern, startRow + 2, startCol );
    setPatternValue(pattern, startRow + 3, startCol );
    setPatternValue(pattern, startRow + 4, startCol );
    setPatternValue(pattern, startRow , startCol );
    setPatternValue(pattern, startRow + 5, startCol );
    setPatternValue(pattern, startRow + 5, startCol + 1);
   setPatternValue(pattern, startRow + 1, startCol + 4);
    setPatternValue(pattern, startRow + 2, startCol + 3);
    setPatternValue(pattern, startRow + 2, startCol + 4);
    setPatternValue(pattern, startRow + 3, startCol + 3);
    setPatternValue(pattern, startRow + 5, startCol + 3);
    setPatternValue(pattern, startRow + 4, startCol + 4);

    setPatternValue(pattern, startRow + 5, startCol + 2);
  }

  else if (c == 'L') {
    setPatternValue(pattern, startRow, startCol);
    setPatternValue(pattern, startRow + 1, startCol);
    setPatternValue(pattern, startRow + 2, startCol);
    setPatternValue(pattern, startRow + 3, startCol);
    setPatternValue(pattern, startRow + 4, startCol);
    setPatternValue(pattern, startRow + 5, startCol);
    setPatternValue(pattern, startRow + 5, startCol + 1);
    setPatternValue(pattern, startRow + 5, startCol + 2);
    setPatternValue(pattern, startRow + 5, startCol + 3);
    setPatternValue(pattern, startRow + 5, startCol + 4);
  }

  else if (c == 'O') {
    setPatternValue(pattern, startRow, startCol + 1);
```

OUTCOME

1. Initial Setup:
    - The canvas is set to a size of 1300x800 pixels.
    - The grid size is set to 50 pixels, resulting in a grid of 26 columns and 16 rows.
2. Pattern Initialization:
    - Each character in "BLOOD" is defined by a 2D boolean array where true indicates the placement of an EyeModule. The patterns for each character are manually defined to visually represent the character.
    - The background (areas not part of "BLOOD") is filled with HeartModule and DropModule instances, chosen randomly.
3. Module Placement:
    - For each grid cell, the code checks if it is part of the "BLOOD" pattern.
    - If it is part of the pattern, an EyeModule is placed.
    - If it is not part of the pattern, a HeartModule or DropModule is placed randomly.
4. Animation and Interaction:
    - DropModule instances fall down the canvas and reappear at the top when they reach the bottom.
    - HeartModule instances can be clicked to break into two halves.
    - EyeModule includes interaction with hovered mouse.

OUTCOME

